SQL Fundamentals — Lecture 2

Filtering · Sorting · Grouping · Pattern Matching · Joins

WHERE · ORDER BY · GROUP BY · HAVING · LIKE · BETWEEN · IN · NOT IN · JOINs

---

Reference Data — Tables Used in This Lecture

All examples in this lecture use the following two tables. Study them carefully — you will need them to understand every query.

Students Table

student_id	first_name	last_name	city	grade	score
1	Sara	Ahmed	Ramallah	A	92
2	Omar	Hassan	Nablus	B+	85
3	Lina	Khalid	Ramallah	A	95
4	Yusuf	Nasser	Hebron	C	71
5	Hana	Samir	Nablus	B	80
6	Tariq	Jaber	Hebron	A	90
7	Rima	Khalil	Ramallah	F	45
8	Faris	Odeh	Nablus	B+	88

Enrollments Table

enrollment_id	student_id	course_name	semester
1	1	Database Systems	Fall 2024
2	1	Web Development	Fall 2024
3	2	Database Systems	Fall 2024
4	3	Data Structures	Fall 2024
5	5	Web Development	Fall 2024
6	6	Database Systems	Fall 2024
7	8	Data Structures	Fall 2024
8	9	Algorithms	Fall 2024

Note: *Student ID 9 in Enrollments does not exist in the Students table — this is intentional and will matter when we learn about JOINs.*

1 WHERE — Filtering Rows

The WHERE clause filters which rows are returned by a query. It comes after FROM and before ORDER BY / GROUP BY.

Syntax

```
SELECT columnsFROM table_nameWHERE condition;
```

1.1 Comparison Operators

Operator	Meaning	Example
=	Equal to	grade = 'A'
!= or <>	Not equal to	city != 'Hebron'
>	Greater than	score > 80
<	Less than	score < 70
>=	Greater than or equal	score >= 90
<=	Less than or equal	score <= 75

1.2 Logical Operators — AND / OR / NOT

SQL — WHERE with AND / OR / NOT

```
-- Students from Ramallah with grade A
SELECT * FROM Students
WHERE city = 'Ramallah' AND grade = 'A';

-- Students from Nablus OR Hebron
SELECT * FROM Students
WHERE city = 'Nablus' OR city = 'Hebron';

-- Students who did NOT get an F
SELECT * FROM Students
WHERE NOT grade = 'F';
```

Note: *Use parentheses to control precedence when mixing AND and OR: (A OR B) AND C*

1.3 WHERE with NULL

SQL — IS NULL / IS NOT NULL

```
-- Find rows where a value is missing
SELECT * FROM Students WHERE city IS NULL;

-- Find rows where a value exists
SELECT * FROM Students WHERE city IS NOT NULL;
```

Key Rules for WHERE

- Text values must be in single quotes: city = 'Ramallah'
- Numbers do not use quotes: score > 80
- Never compare to NULL with = — always use IS NULL or IS NOT NULL
- You can combine multiple conditions with AND, OR, NOT

2 ORDER BY — Sorting Results

ORDER BY sorts the result set by one or more columns. By default the sort is ascending (A→Z, 0→9).

Syntax

```
SELECT columns  FROM table_name  ORDER BY column1 ASC | DESC,
column2 ...;
```

SQL — ORDER BY Examples

```sql
-- Sort students by score highest to lowest
SELECT first_name, last_name, score
FROM Students
ORDER BY score DESC;

-- Sort alphabetically by last name, then first name
SELECT first_name, last_name
FROM Students
ORDER BY last_name ASC, first_name ASC;

-- Filter AND sort together
SELECT first_name, score
FROM Students
WHERE city = 'Ramallah'
ORDER BY score DESC;
```

2.1 Result — ORDER BY score DESC

first_name	last_name	score
Lina	Khalid	95
Sara	Ahmed	92
Tariq	Jaber	90
Faris	Odeh	88
Omar	Hassan	85
Hana	Samir	80
Yusuf	Nasser	71
Rima	Khalil	45

ASC vs DESC

- ASC (default) — smallest to largest, A to Z, oldest to newest
- DESC — largest to smallest, Z to A, newest to oldest
- You can sort by multiple columns: ORDER BY city ASC, score DESC
- ORDER BY always comes after WHERE

3 GROUP BY — Aggregating Data

GROUP BY groups rows that share the same value in a column so you can run aggregate functions (COUNT, SUM, AVG, MAX, MIN) on each group.

Syntax

```
SELECT column, AGG_FUNCTION(column)FROM table_nameGROUP BY
column;
```

3.1 Aggregate Functions

Function	What it does	Example
COUNT(*)	Count number of rows	COUNT(*) → 8
COUNT(col)	Count non-NULL values	COUNT(grade)
SUM(col)	Add all values	SUM(score) → 646
AVG(col)	Calculate the average	AVG(score) → 80.75
MAX(col)	Find the highest value	MAX(score) → 95
MIN(col)	Find the lowest value	MIN(score) → 45

3.2 GROUP BY Examples

SQL — GROUP BY Examples

```
-- How many students are in each city?
SELECT city, COUNT(*) AS total_students
FROM Students
GROUP BY city;

-- Average score per city
SELECT city, AVG(score) AS avg_score
FROM Students
GROUP BY city;

-- Highest score per grade
```

```
SELECT grade, MAX(score) AS top_score
FROM Students
GROUP BY grade
ORDER BY grade ASC;
```

3.3 Results

COUNT(*) per city

city	total_students
Ramallah	3
Nablus	3
Hebron	2

AVG(score) per city

city	avg_score
Ramallah	77.33
Nablus	84.33
Hebron	80.50

> **Note:** *AS gives the column a readable alias name in the result — 'AS total_students' instead of 'COUNT(*)'*

Important Rule

Every column in SELECT that is NOT inside an aggregate function MUST appear in GROUP BY.

✓ SELECT city, COUNT(*) FROM Students GROUP BY city;

✗ SELECT city, first_name, COUNT(*) FROM Students GROUP BY city; -- ERROR!

---

4 HAVING — Filtering Groups

HAVING filters groups after GROUP BY has run. It is like WHERE, but for groups not individual rows.

WHERE vs HAVING — The Key Difference

Clause	Filters	When it runs	Can use aggregates?
WHERE	Individual rows	Before GROUP BY	No
HAVING	Groups of rows	After GROUP BY	Yes

Syntax

```
SELECT column, AGG_FUNCTION(column)FROM table_nameGROUP BY
columnHAVING condition;
```

SQL — HAVING Examples

```
-- Show only cities with MORE than 2 students
SELECT city, COUNT(*) AS total
FROM Students
GROUP BY city
HAVING COUNT(*) > 2;

-- Show only cities where the average score is 80 or above
SELECT city, AVG(score) AS avg_score
FROM Students
GROUP BY city
HAVING AVG(score) >= 80;

-- Combining WHERE and HAVING
-- Cities (excluding Hebron) with more than 1 student
SELECT city, COUNT(*) AS total
FROM Students
WHERE city != 'Hebron'
GROUP BY city
HAVING COUNT(*) > 1;
```

4.1 Query Execution Order

Step	Clause	What Happens
1	FROM	Load the table
2	WHERE	Filter individual rows
3	GROUP BY	Group the remaining rows
4	HAVING	Filter the groups
5	SELECT	Choose which columns to show

6	ORDER BY	Sort the final results

5 LIKE — Pattern Matching

LIKE searches for a pattern inside a text column. It uses two wildcard characters:

Wildcard	Meaning	Example Pattern	Matches
%	Any sequence of zero or more characters	'A%'	Ahmed, Ali, Amal
_	Exactly one character	'_ana'	Hana, Dana, Jana

Syntax

```
SELECT columns  FROM table_name  WHERE column LIKE 'pattern';
```

SQL — LIKE Examples

```
-- Last names that START with 'K'
SELECT * FROM Students WHERE last_name LIKE 'K%';

-- Names that END with 'a'
SELECT * FROM Students WHERE first_name LIKE '%a';

-- Names that CONTAIN 'ar' anywhere
SELECT * FROM Students WHERE first_name LIKE '%ar%';

-- Exactly 4-character first names
SELECT * FROM Students WHERE first_name LIKE '____';

-- NOT LIKE — names that do NOT start with 'S'
SELECT * FROM Students WHERE first_name NOT LIKE 'S%';
```

Tip

- LIKE is case-insensitive in MySQL by default.
- % can replace 0, 1, or many characters.
- Use multiple _ underscores to match an exact character count: '__na' = exactly 4

chars ending in 'na'.

- NOT LIKE excludes rows matching the pattern.

6 BETWEEN — Range Filtering

BETWEEN checks if a value falls within a range — including both the start and end values (it is inclusive on both ends).

Syntax

```
SELECT columns  FROM table_name  WHERE column BETWEEN low_value
AND high_value;
```

SQL — BETWEEN Examples

```
-- Students with scores between 80 and 95 (inclusive)
SELECT first_name, score
FROM Students
WHERE score BETWEEN 80 AND 95;

-- The same query written WITHOUT BETWEEN
SELECT first_name, score
FROM Students
WHERE score >= 80 AND score <= 95;

-- BETWEEN with text (alphabetical range)
SELECT * FROM Students
WHERE first_name BETWEEN 'H' AND 'R';

-- NOT BETWEEN — scores outside 70-89
SELECT first_name, score
FROM Students
WHERE score NOT BETWEEN 70 AND 89;
```

6.1 Result — score BETWEEN 80 AND 95

first_name	score
Sara	92

Lina	95
Hana	80
Tariq	90
Omar	85
Faris	88

> **Note:** *BETWEEN is inclusive: BETWEEN 80 AND 95 includes rows where score = 80 and score = 95.*

---

7 IN and NOT IN — List Matching

IN checks if a value matches any item in a list. NOT IN checks that the value does NOT match any item in the list.

Syntax

```
SELECT columns  FROM table_name  WHERE column IN (value1, value2,
value3, ...);
```

SQL — IN and NOT IN Examples

```
-- Students from Ramallah or Nablus
SELECT * FROM Students
WHERE city IN ('Ramallah', 'Nablus');

-- This is equivalent to (but shorter than):
SELECT * FROM Students
WHERE city = 'Ramallah' OR city = 'Nablus';

-- Students with grades A, A+, or B+
SELECT first_name, grade
FROM Students
WHERE grade IN ('A', 'A+', 'B+');

-- NOT IN — students NOT from Hebron or Nablus
SELECT * FROM Students
WHERE city NOT IN ('Hebron', 'Nablus');

-- IN with numbers — specific student IDs
SELECT * FROM Students
WHERE student_id IN (1, 3, 6);
```

7.1 IN vs OR — When to Use Which

Approach	Example	When to use
OR	city = 'A' OR city = 'B'	2 values — either is fine
IN	city IN ('A','B','C','D')	3+ values — much cleaner
NOT IN	city NOT IN ('A','B')	Excluding a list of values

Warning — NOT IN with NULL

If the list contains NULL, NOT IN returns no results!

```
WHERE city NOT IN ('Hebron', NULL)  -- returns 0 rows!
```

Always make sure your IN list has no NULL values.

8 Combining Clauses — Full SELECT Structure

All the clauses we have learned can be combined in one query. The order they are written must follow this fixed structure:

```
SELECT columnsFROM table_nameWHERE row_conditionGROUP BY
columnHAVING group_conditionORDER BY column ASC | DESC;
```

SQL — Full Combined Query

```
-- Cities with avg score >= 80, sorted highest to lowest
-- excluding students with grade F
SELECT city,
       COUNT(*)    AS total_students,
       AVG(score)  AS avg_score,
       MAX(score)  AS top_score
FROM Students
WHERE grade != 'F'
GROUP BY city
HAVING AVG(score) >= 80
ORDER BY avg_score DESC;
```

9 JOINs — Combining Two Tables

A JOIN combines rows from two or more tables based on a related column (usually a foreign key linking to a primary key). There are 4 types of JOINs.

9.0 Why Do We Need JOINs?

Our data is split across two tables: Students and Enrollments. To answer questions like "which courses is Sara enrolled in?" we need to combine them. The link between the tables is the student_id column.

Question	Needs JOIN?	Tables Involved
What is Sara's grade?	No — single table	Students
Which courses is Sara enrolled in?	Yes	Students + Enrollments
How many students are enrolled per course?	Yes	Students + Enrollments
Which students have NO enrollments?	Yes (LEFT JOIN)	Students + Enrollments

9.1 INNER JOIN — Matching Rows Only

Returns only the rows where there is a match in BOTH tables. Rows with no match in either table are excluded.

INNER JOIN

A ●●● ∩ ●●● B

Only matching rows

Students ∩ Enrollments

Only students who ARE enrolled.

Students with NO enrollment: excluded.

```
SELECT columnsFROM TableAINNER JOIN TableB  ON TableA.id =
TableB.id;
```

SQL — INNER JOIN

```
-- Show each student's name alongside their enrolled courses
SELECT s.first_name, s.last_name, e.course_name, e.semester
FROM Students s
INNER JOIN Enrollments e  ON s.student_id = e.student_id;
```

Result — INNER JOIN

first_name	last_name	course_name	semester
Sara	Ahmed	Database Systems	Fall 2024
Sara	Ahmed	Web Development	Fall 2024
Omar	Hassan	Database Systems	Fall 2024
Lina	Khalid	Data Structures	Fall 2024
Hana	Samir	Web Development	Fall 2024
Tariq	Jaber	Database Systems	Fall 2024
Faris	Odeh	Data Structures	Fall 2024

Note: *Yusuf, Rima, and the enrollment for student_id 9 (non-existent) are all excluded because they have no match in the other table.*

9.2 LEFT JOIN — All Rows from the Left Table

Returns ALL rows from the LEFT (first) table, and matching rows from the right table. If there is no match in the right table, NULL is shown for those columns.

LEFT JOIN

A ●●●●● ∩ — — B

All of A + matches from B

ALL Students + matching Enrollments

Students with no enrollment: included (NULLs in course columns).

```
SELECT columnsFROM TableALEFT JOIN TableB  ON TableA.id =
TableB.id;
```

SQL — LEFT JOIN

```
-- ALL students, including those with no enrollment
SELECT s.first_name, s.last_name, e.course_name
FROM Students s
LEFT JOIN Enrollments e  ON s.student_id = e.student_id;

-- Useful: find students who are NOT enrolled in anything
SELECT s.first_name, s.last_name
FROM Students s
LEFT JOIN Enrollments e  ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

Partial Result — LEFT JOIN (notice NULLs)

first_name	last_name	course_name
Sara	Ahmed	Database Systems
Sara	Ahmed	Web Development
Omar	Hassan	Database Systems
Lina	Khalid	Data Structures
Yusuf	Nasser	NULL ← not enrolled
Hana	Samir	Web Development
Tariq	Jaber	Database Systems
Rima	Khalil	NULL ← not enrolled
Faris	Odeh	Data Structures

9.3 RIGHT JOIN — All Rows from the Right Table

Returns ALL rows from the RIGHT (second) table, and matching rows from the left table. If there is no match in the left table, NULL is shown.

RIGHT JOIN

A — — ∩ ●●●●● B

All of B + matches from A

ALL Enrollments + matching Students

Enrollment for student_id 9 (non-existent): included (NULLs in student columns).

SQL — RIGHT JOIN

```
-- ALL enrollments, including those with no matching student
SELECT s.first_name, s.last_name, e.course_name
FROM Students s
RIGHT JOIN Enrollments e  ON s.student_id = e.student_id;
```

Note: *RIGHT JOIN is less commonly used. Most developers rewrite it as a LEFT JOIN by swapping the table order. Both achieve the same result.*

9.4 FULL OUTER JOIN — All Rows from Both Tables

Returns ALL rows from both tables. Where there is no match, NULL is shown for the missing side. You get everything from both tables combined.

FULL OUTER JOIN

A ●●●●● ∩ ●●●●● B

Everything from A and B

ALL Students + ALL Enrollments

Unmatched rows from BOTH sides appear, with NULLs filling the gaps.

SQL — FULL OUTER JOIN

```
-- All students AND all enrollments — even unmatched ones
SELECT s.first_name, s.last_name, e.course_name
FROM Students s
FULL OUTER JOIN Enrollments e  ON s.student_id = e.student_id;

-- MySQL does not support FULL OUTER JOIN directly.
-- Simulate it with UNION:
SELECT s.first_name, s.last_name, e.course_name
FROM Students s  LEFT JOIN Enrollments e ON s.student_id = e.student_id
UNION
SELECT s.first_name, s.last_name, e.course_name
FROM Students s  RIGHT JOIN Enrollments e ON s.student_id =
e.student_id;
```

9.5 JOIN Comparison Summary

JOIN Type	Returns	Left-only rows?	Right-only rows?	Use when...
INNER JOIN	Matching rows only	No	No	You only want rows with data in both tables
LEFT JOIN	All left + matched right	Yes	No	You want all rows from left, match if possible
RIGHT JOIN	All right + matched left	No	Yes	You want all rows from right, match if possible
FULL OUTER JOIN	All rows from both tables	Yes	Yes	You want every row, matched or not

Note: *In practice INNER JOIN and LEFT JOIN are the most commonly used. Master these two first.*

9.6 Combining JOINs with Other Clauses

SQL — JOIN + GROUP BY + WHERE

```sql
-- How many students are enrolled per course?
SELECT e.course_name, COUNT(s.student_id) AS enrolled_count
FROM Enrollments e
INNER JOIN Students s  ON e.student_id = s.student_id
GROUP BY e.course_name
ORDER BY enrolled_count DESC;

-- Students from Ramallah and their courses
SELECT s.first_name, s.city, e.course_name
FROM Students s
INNER JOIN Enrollments e  ON s.student_id = e.student_id
WHERE s.city = 'Ramallah';
```

10 Quick Reference — Today's Lecture

Clause / Keyword	Purpose	Quick Example
WHERE	Filter rows before grouping	WHERE score > 80
ORDER BY	Sort results ASC or DESC	ORDER BY score DESC
GROUP BY	Group rows for aggregation	GROUP BY city
HAVING	Filter groups after GROUP BY	HAVING COUNT(*) > 2
LIKE	Pattern match with % and _	WHERE name LIKE 'S%'
BETWEEN	Inclusive range filter	WHERE score BETWEEN 70 AND 90
IN	Match against a list of values	WHERE city IN ('A','B')
NOT IN	Exclude a list of values	WHERE city NOT IN ('A')
INNER JOIN	Rows matching in both tables	A INNER JOIN B ON A.id=B.id
LEFT JOIN	All left rows + right matches	A LEFT JOIN B ON A.id=B.id
RIGHT JOIN	All right rows + left matches	A RIGHT JOIN B ON A.id=B.id
FULL OUTER JOIN	All rows from both tables	A FULL OUTER JOIN B ON A.id=B.id

11 Practice Exercises

Use the Students and Enrollments tables from Section 1 to answer the following questions. Write the full SQL query for each.

Part A — Filtering & Sorting

- 1. Select all students from Nablus, sorted by score descending.
- 2. Select students whose score is between 75 and 90 (inclusive).
- 3. Select students whose first name starts with any letter between 'L' and 'T'.
- 4. Select students from Ramallah or Hebron using IN.
- 5. Select students who are NOT from Nablus using NOT IN.

Part B — Grouping & Aggregation

- 6. Count how many students are in each city.
- 7. Find the average and maximum score per grade.
- 8. Show only grades where the average score is above 85 (use HAVING).
- 9. Count students per city, but exclude students with grade F first (use WHERE + GROUP BY).

Part C — JOINs

- 10. Show each student's full name and the courses they are enrolled in (INNER JOIN).
- 11. Show ALL students, with their course if enrolled, or NULL if not (LEFT JOIN).
- 12. Find students who are NOT enrolled in any course (LEFT JOIN + IS NULL).
- 13. Count how many students are enrolled per course (JOIN + GROUP BY).
- 14. Show students from Ramallah and their enrolled courses (JOIN + WHERE).

Bonus Challenge — Combine Everything

- 15. For each city, count how many students are enrolled in at least one course. Show only cities with 2 or more enrolled students. Sort by count descending. (Hint: you need a JOIN + GROUP BY + HAVING + ORDER BY)